Part 2: MLOps System Walkthrough & Verification Report

Part 2: Student Performance & Academic Risk Prediction System (MLOps Platform)

Executive Summary

Part 2 has been successfully implemented, extending the existing Part 1 data engineering foundation into a reproducible, end-to-end **MLOps Platform**. The system provides real-time early warning failure risk prediction (*G3 < 10*), multi-model experiment tracking via MLflow, containerized serving with FastAPI and Docker, interactive inference and drift monitoring in Streamlit, Population Stability Index (PSI) drift detection, and an automated champion-vs-challenger retraining workflow.

■ **Note:** [!IMPORTANT]

■ **Note: Key Quality Highlights:**

■ **Note:** - **Zero Target Leakage:** Predictors strictly exclude *G2*, *G3*, final grades, average grades, and post-outcome labels.

■ **Note:** - **Champion Model: XGBoost Classifier** selected with **88.96% Accuracy**, **84.85% Validation Recall**, **0.7671 Validation F1**, and **0.9329 ROC-AUC**.

■ **Note:** - **100% Test Pass Rate: 37 automated unit and integration tests** passing across Part 1 and Part 2 (**`python -m pytest tests/ -v`**).

■ **Note:** - **Serving Readiness:** REST endpoints running on FastAPI with Swagger docs at http://localhost:8000/docs and seamless local fallback in the Streamlit UI.

1. End-to-End System Architecture

```
flowchart TD
    subgraph Part 1: Data Engineering
        A["Raw Data Ingestion<br/>(student-mat.csv, student-por.csv)"] --> B["Multi-Stage Schema Validation<br/>& Quarantine Logging"]
        B --> C["Relational PostgreSQL Warehouse<br/>& Analytical Data Mart (1,028 rows)"]
        C --> D["Airflow ETL DAG<br/>(student_pipeline.py)"]
    end

    subgraph Part 2: MLOps Pipeline
        C --> E["Leak-Free Feature Engineering<br/>(ColumnTransformer: Scaler + OHE)"]
        E --> F["Stratified Splitting<br/>(Train: 719, Val: 154, Test: 155)"]
        F --> G["Model Benchmarking & MLflow Tracking<br/>(Logistic Regression, RF, XGBoost)"]
        G --> H["Model Registry & Governance<br/>(Champion: XGBoost v1.0.0)"]
        H --> I["FastAPI Microservice (Port 8000)<br/>(/health, /model-info, /predict, /predict/batch, /metrics)"]
        I --> J["Dockerized Stack<br/>(Postgres + API + Streamlit)"]
        H --> K["Streamlit Portal (7 Views)<br/>View 6: ML Prediction | View 7: Monitoring & Drift"]
        K --> L["Drift Detection Engine<br/>(Population Stability Index / PSI)"]
        L --> M["Champion-vs-Challenger<br/>Automated Retraining Loop"]
        M -. Promoted .-> H
    end
```

2. Prediction Target Formulation & Zero Leakage

• **Target Variable: `failure_risk`** *\in \{0, 1\}*

• *1 = \text{Academic Failure Risk} \ (G3 < 10)*

• *0 = \text{Passing Trajectory} \ (G3 \ge 10)*

• **Defensibility:** In Portuguese secondary education, 10 on a 20-point scale is the universal legal passing threshold. Predicting this outcome prior to final assessments allows institutions to deploy academic tutoring, study habit coaching, and counseling before failure occurs.

• **Dataset Fingerprint (`data/reference/dataset_fingerprint.json`):**

• Total Validated Records: 1,028

• Passing (**0**): 806 records (78.4%)

• At-Risk (**1**): 222 records (21.6%)

• Class Imbalance: *\approx 3.64:1* (counteracted via balanced class weights and positive class scaling).

- **Strict Leakage Prevention Protocol:**
- **Excluded:** *G2* (Period 2), *G3* (Period 3), **`final_grade`**, **`average_grade`**, **`pass_fail`**, **`performance_category`**, **`grade_improvement`**, **`risk_status`**, and **`student_id`**.
- **Selected Features (16 predictors):**
- Numerical (6): **`g1`**, **`age`**, **`absences`**, **`attendance_rate`**, **`studytime`**, **`failures`**.
- Categorical (10): **`school`**, **`sex`**, **`address`**, **`famsize`**, **`pstatus`**, **`mjob`**, **`fjob`**, **`parent_education`**, **`study_time_category`**, **`subject`**.

3. Empirical Model Benchmarking & MLflow Results

All models were evaluated on identical stratified splits (Train: 719, Val: 154, Test: 155) with random seed 42.

Model Candidate	Split	Accuracy	Precision	Recall	F1-Score	ROC-AUC	PR-AUC
Logistic Regression (Baseline)	Validation	86.36%	0.6579	0.7576	0.7042	0.9256	0.7381
	Test	84.52%	0.5957	0.8235	0.6914	0.9409	0.7562
Random Forest Classifier	Validation	87.66%	0.7188	0.6970	0.7077	0.9276	0.7602
	Test	87.10%	0.6579	0.7353	0.6944	0.9186	0.7411
XGBoost Classifier (Champion)	**Validation**	**88.96%**	**0.7000**	**0.8485**	**0.7671**	**0.9329**	**0.7651**
	Test	**85.16%**	**0.6279**	**0.7941**	**0.7013**	**0.9220**	**0.7534**

Champion Model Selection Rationale

In student dropout and academic failure early warning, **Recall is prioritized** over raw accuracy. Missing a failing student (False Negative) deprives them of academic intervention, whereas flagging a passing student (False Positive) merely leads to check-in advisory support.

- **XGBoost Classifier** was crowned Champion because it achieved the **highest Recall (84.85%)** while maintaining the **highest F1-score (0.7671)** and **ROC-AUC (0.9329)** on validation data, maintaining strong generalisation on the unbiased test set (Recall 79.41%, ROC-AUC 0.9220).

4. Evaluation Plots & Lineage Artifacts

Artifact plots saved to reports/:

1. **Confusion Matrix:** **`cm_xgboost_classifier.png`**
- On Test Split: 27 True Positives (At-Risk caught), 105 True Negatives, only 7 missed students.
2. **ROC Curve:** **`roc_xgboost_classifier.png`**
- AUC = 0.922 showing discrimination power.
3. **Precision-Recall Curve:** **`pr_xgboost_classifier.png`**
- Average Precision = 0.753 substantially outperforming baseline frequency (0.216).

5. FastAPI Prediction Microservice & Swagger UI

Implemented in **`api/main.py`**:

Endpoint	Method	Description	Sample Output / Status
`/health`	**`GET`**	Service readiness and model loading status	**`{"status": "ok", "model_loaded": true, "model_name": "XGBoost Classifier", "model_version": "v1.0.0"}`**
`/model-info`	**`GET`**	Champion metadata, hyperparameters, metrics	Full audit governance specification

`/predict`	`POST`	Single student failure risk prediction	`{"prediction": 1, "risk_status": "At-Risk", "risk_score": 0.8142, "risk_level": "High Risk", ...}`
`/predict/batch`	`POST`	Vectorized multi-student roster scoring	Roster aggregate metrics + per-student predictions
`/metrics`	`GET`	Operational Prometheus-style metrics	Request count, alert count, average latency (ms)

6. Streamlit Dashboard Integration (Views 6 & 7)

`dashboard/app.py` was seamlessly expanded to **7 interactive analytical views**:

- **View 1:** Executive KPI Summary & Grade Distributions.
- **View 2:** Attendance Impact, Scatter Correlation & Chronic Absenteeism.
- **View 3:** Subject Performance Comparison (Mathematics vs Portuguese).
- **View 4:** Socio-Demographic & Parental Education Success Indicators.
- **View 5:** Student Diagnostic Dossier & Academic Early Warning System.
- **View 6: ■ Real-Time Academic Risk Prediction (ML Inference):**
- Interactive student profile configuration form.
- Risk Gauge Plot visualizing predicted failure probability.
- Contextual pedagogical action plan (Tutoring, study habit coaching).
- One-click batch cohort scoring across the active filtered data mart.
- **View 7: ■ MLOps Model Governance & Drift Diagnostics:**
- Active champion specifications and MLflow experiment details.
- Empirical model comparison table across candidate algorithms.
- Embedded evaluation artifacts (Confusion Matrix, ROC Curve, PR Curve).
- Live Population Stability Index (PSI) feature drift table with status badges.
- Documented model retraining policy and promotion rules.

7. Monitoring & Retraining Governance

1. **Data Quality Validation (`src/monitoring/data_quality.py`):**
- Validates input features against numerical bounds ($0 \le G1 \le 20$, $0 \le \text{attendance} \le 100\%$) and known categorical levels.
2. **Feature Drift Engine (`src/monitoring/drift.py`):**
- Uses **Population Stability Index (PSI)** to compare inference traffic against **`reference_features.parquet`**.
- $\text{PSI} < 0.10$: Stable distribution (No Drift).
- $0.10 \le \text{PSI} < 0.20$: Moderate distribution shift (Warning).
- $\text{PSI} \ge 0.20$: Critical distribution shift (Triggers Retraining Alert).
3. **Automated Retraining Loop (`src/training/retrain.py`):**
- Trains challenger candidates on updated analytical data.
- **Promotion Criteria:**
- Challenger must improve Validation F1 over Champion by $\Delta \ge 0.005$.
- Validation Recall must remain ≥ 0.70 (Safety floor).
- Updates **`models/model_registry.json`** with version increment (**`v1.1.0`**).

8. Test Suite Verification (37 Passed)

```
python -m pytest tests/ -v
```

```
============================= test session starts ==============================
platform win32 -- Python 3.12.6, pytest-9.1.1
collected 37 items

tests/test_api.py::test_health_endpoint PASSED                           [  2%]
tests/test_api.py::test_model_info_endpoint PASSED                       [  5%]
tests/test_api.py::test_predict_single_passing_student PASSED            [  8%]
tests/test_api.py::test_predict_single_at_risk_student PASSED            [ 10%]
tests/test_api.py::test_predict_validation_error PASSED                  [ 13%]
tests/test_api.py::test_predict_batch PASSED                             [ 16%]
tests/test_api.py::test_metrics_endpoint PASSED                          [ 18%]
tests/test_features.py::test_no_target_leakage_in_features PASSED        [ 21%]
tests/test_features.py::test_dataset_fingerprinting PASSED               [ 24%]
tests/test_features.py::test_preprocessor_fitting_and_transformation PASSED [ 27%]
tests/test_features.py::test_stratified_data_split_integrity PASSED      [ 29%]
tests/test_ingestion.py::test_detect_delimiter PASSED                    [ 32%]
tests/test_ingestion.py::test_ingest_file_missing_raises PASSED          [ 35%]
tests/test_ingestion.py::test_ingest_existing_raw_data PASSED            [ 37%]
tests/test_monitoring.py::test_data_quality_validator_valid_record PASSED [ 40%]
tests/test_monitoring.py::test_data_quality_validator_out_of_bounds PASSED [ 43%]
tests/test_monitoring.py::test_numerical_psi_identical_distribution PASSED [ 45%]
tests/test_monitoring.py::test_numerical_psi_shifted_distribution PASSED [ 48%]
tests/test_monitoring.py::test_categorical_psi_frequency_shift PASSED    [ 51%]
tests/test_monitoring.py::test_drift_detector_end_to_end PASSED          [ 54%]
tests/test_retrain.py::test_promotion_when_challenger_improves_f1 PASSED [ 56%]
tests/test_retrain.py::test_rejection_when_challenger_recall_violates_safety_floor PASSED [ 59%]
tests/test_retrain.py::test_rejection_when_challenger_has_inferior_f1 PASSED [ 62%]
tests/test_training.py::test_metric_calculation_correctness PASSED       [ 64%]
tests/test_training.py::test_champion_model_artifacts_exist PASSED       [ 67%]
tests/test_training.py::test_champion_model_inference PASSED             [ 70%]
tests/test_training.py::test_model_registry_governance_schema PASSED     [ 72%]
tests/test_transformation.py::test_map_performance_category PASSED       [ 75%]
tests/test_transformation.py::test_calculate_risk_status PASSED          [ 78%]
tests/test_transformation.py::test_transformation_pipeline_features PASSED [ 81%]
tests/test_validation.py::test_valid_row_passes PASSED                   [ 83%]
tests/test_validation.py::test_invalid_grade_range PASSED                [ 86%]
tests/test_validation.py::test_invalid_categorical_values PASSED         [ 89%]
tests/test_validation.py::test_negative_absences PASSED                  [ 91%]
tests/test_validation.py::test_missing_values PASSED                     [ 94%]
tests/test_validation.py::test_validate_dataframe_quarantine PASSED      [ 97%]
tests/test_validation.py::test_duplicate_student_quarantine PASSED       [100%]

============================== 37 passed in 5.31s ==============================
```

9. Quick-Start Execution Commands

Task	Command
Run Feature Pipeline & Training	**`run_training.bat`** or **`python src/training/train.py`**
Launch MLflow UI	**`run_mlflow_ui.bat`** or **`mlflow ui --backend-store-uri sqlite:///mlruns.db --port 5000`**
Launch FastAPI Server	**`run_api.bat`** or **`python -m uvicorn api.main:app --host 0.0.0.0 --port 8000 --reload`**
Launch Streamlit Dashboard	**`run_dashboard.bat`** or **`streamlit run dashboard/app.py`**
Run Full Test Suite	**`python -m pytest tests/ -v`**
Run with Docker Compose	**`docker-compose up --build`**